FixEdu

School Facility Condition Reporting & Repair Tracking Portal

Product Requirements Document (PRD)

Author: Shantanu Pabba

Organization: Matrusri Engineering College

Email: cse23733018@matrusri.edu.in

Version: 1.0.0

Date: July 14, 2026

Status: Completed / Approved

Table of Contents

1. Executive Summary & Goals

1.1 Problem Statement

Educational institutions often struggle with managing facilities, classrooms, labs, and equipment repair lifecycles. Reports of broken infrastructure (flickering lights, faulty HVAC, water leakage) are commonly submitted via paper receipts or unmonitored emails, resulting in delays, lack of accountability, and poor documentation for administrative budgets.

1.2 Product Vision

FixEdu is designed as a centralized, cloud-native full-stack application that provides immediate transparency into the physical condition of a campus. It facilitates real-time communication between those who detect issues (students and teachers), those who manage priorities (administrators), and those who perform repairs (maintenance technicians).

1.3 Core Business Goals

- **Minimize Repair Delays**: Provide an automated status tracking system to move tasks quickly from assignment to completion.
- **Documented Audit Trail**: Capture all transitions in state history log entities for compliance and budgeting records.
- **Prevent Resource Waste**: Limit double-reporting by maintaining a searchable history page.

2. Detailed User Personas & Workflows

2.1 Student / Teacher (Reporters)

Scenario: Shantanu (a student) enters the Chemistry Lab and discovers that a water faucet is leaking, causing water to pool on the floor.

- Shantanu registers/logs in to the portal using his phone.
- He navigates to the "Report Issue" page, fills out the form, selects "Chemistry Lab" from the dropdown list, and takes 3 pictures of the leaking faucet.
- Upon submission, he receives a confirmation email stating his complaint is logged as "Pending Approval".
- He checks his dashboard metrics or searches his history logs to track updates.

2.2 Administrator (Dispatcher)

Scenario: The System Administrator reviews active complaints on their dashboard.

- The Admin logs in using their secure system admin account.
- The Admin dashboard lists the new "Leaking Faucet" complaint under the "Pending Assignment" backlog.
- The Admin clicks the "Assign Staff" button, opens a modal listing all available Maintenance Staff users, and selects "SAM" to handle the work.
- This action changes the ticket status to "Assigned" and sends notification emails to both SAM and Shantanu.

2.3 Maintenance Staff (Technician)

Scenario: SAM receives an email about a leaking faucet in the Chemistry Lab.

- SAM logs in to the portal on his mobile browser and navigates to the "Assigned Tasks" card queue.
- He locates the "Leaking Faucet" card and clicks **"Start Work"**, which updates the status to "In Progress".

- Once the leak is repaired, SAM clicks **"Mark Completed"**, which opens a modal. He enters resolution notes ("Replaced faucet washer") and uploads a completion photo.
- He submits the form, which updates the status to "Resolved". Shantanu receives an email notification with the completion remarks.

3. Functional Requirements Breakdown

3.1 Requirement Priorities (MoSCoW Matrix)

Priority	Module / Requirement	Description
Must Have	Role-based JWT Authentication	Secure password hashing, token validation, and role redirection.
Must Have	Issue Reporting & Proof Uploads	Form submission with multipart/form-data images using Multer.
Must Have	Admin Allocation Flow	Modal selection to route pending issues to specific staff IDs.
Should Have	Live Notifications Drawer	Top bar dropdown checklist containing unread status alerts.
Should Have	Automated Email Updates	SMTP triggers using Nodemailer to dispatch status changes to users.
Could Have	Pagination & Real-Time Query Search	Server-side page offsets and case-insensitive regex query filters.

4. Security & Compliance Architecture

The backend application implements security middleware packages to prevent common web vulnerabilities:

- **Helmet Header Protection**: Sets various HTTP response headers to secure Express against vulnerabilities like clickjacking and XSS.
- **Rate Limiting**: Uses `express-rate-limit` to limit API calls to 100 requests per 15 minutes per IP address to mitigate brute-force and DoS attacks.
- **Data Sanitization**: Implements `express-mongo-sanitize` to automatically strip keys starting with `$` or containing `.`, preventing NoSQL Injection attacks.

- **Token Expiration**: Configures JSON Web Tokens (JWT) with a 30-day expiration period.

5. Database Design & Mongoose Schemas

The system uses Mongoose schemas on MongoDB Atlas. Relations are structured using MongoDB ObjectIds to link documents.

5.1 User Schema

```
{
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['Student', 'Teacher', 'Maintenance Staff', 'Admin'], d
  isVerified: { type: Boolean, default: false },
  resetPasswordToken: String,
  resetPasswordExpire: Date
}
```

5.2 Facility Schema

```
{
  name: { type: String, required: true, unique: true, trim: true },
  building: { type: String, required: true, trim: true },
  locationDetails: { type: String },
  description: { type: String },
  status: { type: String, enum: ['Operational', 'Under Maintenance', 'Requires Attent
}
```

5.3 Complaint Schema

```
{
  reporter: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  facility: { type: mongoose.Schema.Types.ObjectId, ref: 'Facility', required: true }
  title: { type: String, required: true },
  description: { type: String, required: true },
```

```
  severity: { type: String, enum: ['Low', 'Medium', 'High', 'Critical'], default: 'Me
  status: { type: String, enum: ['Pending Approval', 'Assigned', 'In Progress', 'Reso
  images: [{ type: String }],
  assignedTo: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
  completionDetails: {
    notes: { type: String },
    images: [{ type: String }],
    completedAt: { type: Date }
  }
}
```

6. REST API Endpoint Specifications

All endpoints are prefixed with `/api`. Standard JSON envelopes are returned for all API requests.

6.1 Authentication Module

- `POST /auth/register`: Registers a user. Expects name, email, password, and role. Returns signed JWT token and user profile object.
- `POST /auth/login`: Validates credentials. Returns signed JWT token and user profile object.
- `POST /auth/forgot-password`: Generates a temporary hashed password reset token and dispatches an email link to the user.
- `PUT /auth/reset-password/:resettoken`: Accepts a new password, validates the token expiry, and updates the user's password.

6.2 Complaints & Work Management Module

- `POST /complaints`: Files a complaint. Expects title, description, facility, and severity as multipart/form-data. Accepts up to 5 image uploads. (Reporters only)
- `GET /complaints/my`: Retrieves complaints submitted by the logged-in user. Supports query filters: `?page=1&limit=10&status=Resolved&search=AC`.
- `GET /complaints/assigned`: Retrieves complaints assigned to the logged-in Maintenance Staff member. (Maintenance Staff only)
- `PUT /complaints/:id/status`: Updates task status. Accepts status and notes. For "Resolved" status, supports uploading post-repair verification images. (Maintenance Staff/Admin only)

6.3 Admin Panel Module

- `GET /admin/analytics`: Compiles database metrics, status breakdowns, and monthly reporting frequencies.
- `GET /admin/users`: Returns list of all registered users.
- `PUT /admin/users/:id`: Modifies user roles or toggles verification status.

- `PUT /admin/complaints/:id/assign`: Assigns a ticket to a staff member. Expects a body payload with `staffId`.

7. UI/UX Design Systems & Navigation Architecture

7.1 Dark Aesthetics & Visual System

The system uses a dark interface with curated colors to provide a premium look:

- **Backgrounds**: `#0a0b10` (primary canvas), `#121420` (secondary cards), `#1a1d30` (interactive inputs/dropdowns).
- **Brand Gradient**: Linear text gradient from `#ffffff` to `#a5b4fc`.
- **Interactive Indicators**: Purple glows (`#6366f1` with opacity overlays) used to emphasize active states, form focus, and interactive clicks.

7.2 Layout Structures

The portal implements two layouts: **AuthLayout** (centered authentication panels) and **DashboardLayout** (collapsible left-hand sidebar menu, sticky header with notification bell, and main scrollable content area).

8. System Performance, Scaling, & Future Roadmaps

8.1 Performance Optimizations

- **Query Pagination**: Implemented server-side paginated queries on Mongoose models using `skip()` and `limit()` to prevent database bottlenecks.
- **Index Optimization**: Configured database indexes on the MONGODB cluster for frequently queried fields (e.g. `email`, `reporter`, `status`).
- **Lazy Loading & Code Splitting**: Designed components to load asynchronously to reduce initial bundle download sizes on low-bandwidth clients.

8.2 Strategic Growth Roadmap

1. **Milestone 1 (Immediate)**: Launch the system on Vercel and Render using SMTP Mailtrap templates for student-staff integration.

2. **Milestone 2 (Medium Term)**: Integrate WebSockets via Socket.io to support real-time browser notifications for online staff.
3. **Milestone 3 (Long Term)**: Implement campus map pins to allow users to pin the exact coordinates of facility damage.